

Gaming at Warp Speed: Improving Aiming with Late Warp

Ben Boudaoud
bboudaoud@nvidia.com
NVIDIA
USA

Pyarelal Knowles
pknowles@nvidia.com
NVIDIA
USA

Joohwan Kim
sckim@nvidia.com
NVIDIA
USA

Josef Spjut
jspjut@nvidia.com
NVIDIA
USA

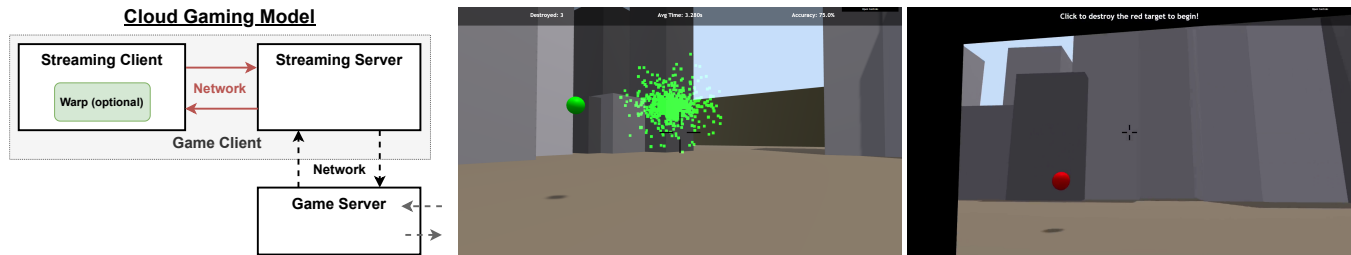


Figure 1: (left) In the cloud gaming paradigm, network latency is added to the game client, resulting in worse aiming performance from players. Late warp, a technique used to prevent simulator sickness in VR, can be applied to first person shooter (FPS) games to mitigate this latency penalty. Using a web-based FPS game (middle), players can test their skill against latency, as well as with late warp correction to see how much late warp helps, even when a naive implementation adds significant guard band artifacts (right). SIGGRAPH virtual conference attendees can run the web app for themselves at home.

ABSTRACT

Latency can make all the difference in competitive online games. Late warp is a class of techniques used in VR that can reduce latency in FPS games as well. Prior work has demonstrated these techniques can recover most of the player performance lost to computer or network latency. Inspired by work demonstrating the usefulness of late warp as a potential solution to FPS latency, we provide an interactive demonstration, playable in a web browser, that shows how much latency limits aiming performance, and how late warp can help.

CCS CONCEPTS

• Human-centered computing → Pointing devices; User models; User studies; • Applied computing → Computer games.

KEYWORDS

pointing devices, mouse, latency, first person targeting, games

ACM Reference Format:

Ben Boudaoud, Pyarelal Knowles, Joohwan Kim, and Josef Spjut. 2021. Gaming at Warp Speed: Improving Aiming with Late Warp. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Emerging Technologies (SIGGRAPH '21 Emerging Technologies)*, August 9–13, 2021, Virtual Event, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3450550.3465347>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGGRAPH '21 Emerging Technologies, August 9–13, 2021, Virtual Event, USA

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8364-6/21/08...\$15.00

<https://doi.org/10.1145/3450550.3465347>

1 INTRODUCTION

In a computer system, latency is the time from an event until an affiliated result. For interactive graphics applications, a primary example is the time from an input until the results of that input are seen by the user. When visual output is a core part of the interaction this latency is arguably most important to the user. In this work, we focus on latency in first person shooter (FPS) video games, or other applications that use a similar control scheme of moving the mouse to rotate an egocentric camera view. FPS games are interesting in particular, as their competitive nature pits human against human, where the most skilled player will win. This competition encourages players to practice and refine their skill to the point that the computer interface becomes an extension of their hands, not unlike the way a paint brush is an extension of the hand for a skilled painter, or heavy machinery is an extension of the human operator. Furthermore, in recent years competitive esports has established a market for the most skilled professional gamers to compete for prizes, and to produce entertainment for spectators. This growth in expert use of interactive computer graphics is an exciting opportunity for new computer graphics and interaction techniques.

One such class of techniques are *late warps*, described at greater length in Section 3. Using late warps, some interaction delays are reduced, and cloud-rendered graphics can be brought (temporally) closer to the user. Armed with this understanding, users can experience the benefits of late warp through the interactive web application described in Section 4. This work is the first simple, interactive demonstration of the impacts of latency on FPS aiming in cloud gaming and use of a rotation-only late warp for mitigating these effects, which has been made widely available for anybody to experience in their web browser¹.

¹Available at <https://nvlabs.github.io/latewarp-s2021/>

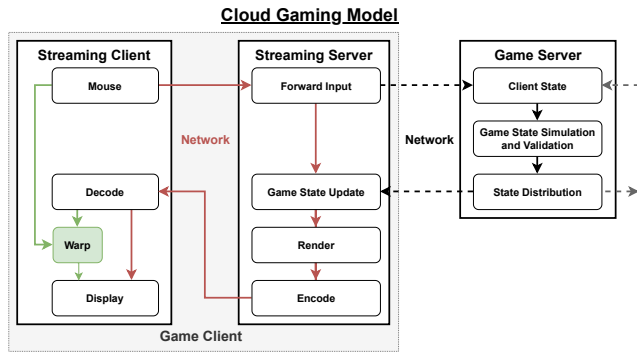


Figure 2: Cloud gaming can add significant latency to the user interaction loop by putting a network connection in the middle of the game client.

2 THE LATENCY PROBLEM

When users talk about latency in computer systems, they are often referring to communication, or round-trip, time in a wide area network. This is common as the world wide web allows computers to connect to each other over large distances, but as a result often creates a majority of the latency experienced by users. Wide area networks may be poorly organized or congested, and electrical or optical signals take time to propagate through copper or fiber links. Today, the speed of light provides an effective lower bound on network latency, assuming all other problems can be solved, implying transmission time halfway around the earth cannot be less than about 70 ms. Given this physical limit, it is impressive that network connections between continents rarely exceed 200 ms.

Another, often taken for granted, but critically important latency is the input-to-output, or *end-to-end* latency of a system. This latency defines the time from a user providing an input (e.g., a mouse movement) to the time the system updates its visual output based on this input (e.g., rotates the view direction). While network latency has come to dominate most discussions in online play, local end-to-end latency can have more immediate impacts on player performance [Liu et al. 2021].

Cloud gaming applications like GeForce Now, Google Stadia, Amazon Luna, PlayStation Now, and Microsoft Project xCloud all provide users the ability to offload part of a game application, that typically runs locally, to cloud servers. As demonstrated in Figure 2, the partitioning of the game client between this streaming server and client adds a network round trip time to the end-to-end latency. Cloud service providers attempt to distribute streaming servers at various data centers in an effort to reduce this round trip latency. Of course latency has always existed between the game client and game server, and a large part of the client-server architecture for multiplayer games works to hide, or reduce, that latency. Unfortunately, these latency-hiding techniques remain exclusive to the streaming server side of the game client in today’s systems.

While cloud gaming latency is significant, and a commonly cited barrier to entry for potential customers, even local gaming performance is impacted by latency. Input devices add delay while handling events and waiting for USB frame timing, and displays’ worst-case LCD pixel transition times are often much higher than

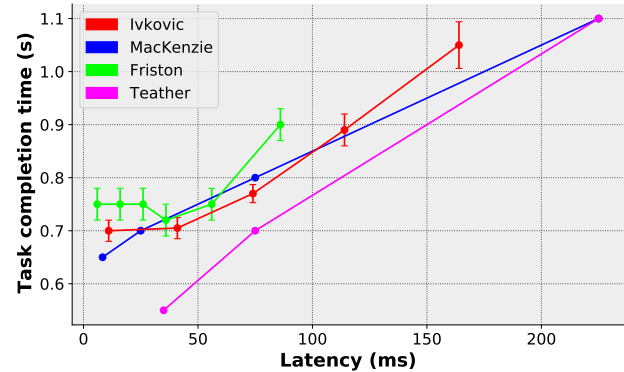


Figure 3: A comparison of results from prior art regarding the impacts of latency on average task completion time.

the advertised gray-to-gray transition time, particularly when using overdrive settings. Recent techniques like NVIDIA Reflex and AMD’s Anti-Lag work to optimize game render-to-display latency. Even so, the best-optimized local gaming systems still experience an average of over 10 ms of latency from input to visual output.

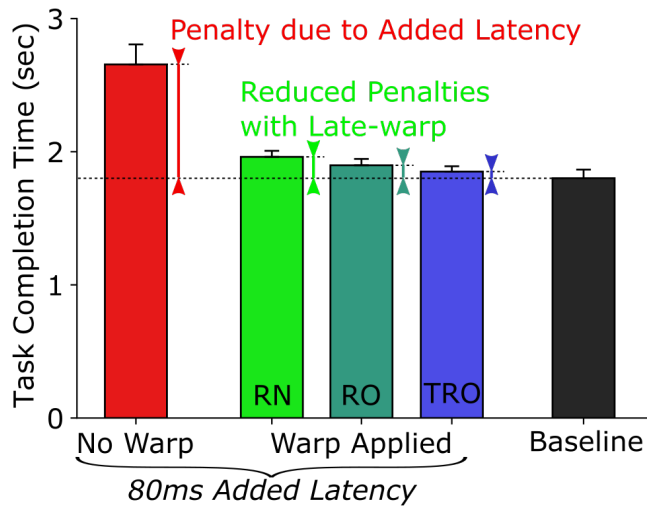
There exists a large body of work studying impacts of latency on the human computer interaction loop, we summarize some relevant previous results in Figure 3. Much of this work has focused on traditional mouse and pointer interfaces where a cursor, or pointer, input is located on a single 2D plane [Cattan et al. 2015; Jota et al. 2013; MacKenzie and Ware 1993; Teather et al. 2009]. In this context, latency increases the amount of time it takes a user to select a target by more than just the latency added to the system. This work has been successful at informing user interface design, as well as stylus and touch interfaces for mobile phones and tablets.

While the 2D pointer use case is important for many interfaces, 3D video games display a 2D projection of a 3D virtual world, and allow the mouse or controller to direct the player’s view within that world. This input modality is distinct in that the 3D view projection is non-linear over view angle, particularly when using wide field of view perspective projections. However, [Looser et al. 2005] showed that 3D targeting task difficulty follows approximately the same Fitts’ Law [Fitts and Peterson 1964] trend as 2D tasks. In this context, recent work [Ivkovic et al. 2015; Spjut et al. 2019a, 2021] has demonstrated the impact of latency on aiming task performance in First Person Shooter (FPS) games, which use perspective projection.

Our demo, described in Section 4, quantifies the impact of 80 ms of added latency on player performance in the FPS context.

3 LATE WARP

Late warps are a family of techniques, popularized by virtual reality (VR) headsets, to reduce the latency of some input on the final image displayed to a user. In the VR context, the most important latency to reduce is that of head motion-to-video output, since any error from latency can cause loss of immersion, simulator sickness, or even loss of balance. While VR commonly uses late warp techniques to reduce latency from head motion, it typically allows for higher latency from controller input or simulation. This is seen as a reasonable trade off because the most important problem to solve is that of motion sickness and/or loss of immersion.



Copyright © 2020 Nvidia.

Figure 4: Average task completion time showing the performance penalty from 80 ms of added latency. Rotation-naive (RN), rotation-oracle (RO) and translation-rotation-oracle (TRO) warps all mitigate this completion time penalty. Figure from [Kim et al. 2020].

More recently, [Kim et al. 2020] proposed the application of late warp to FPS games, particularly for gamers interested in competitive esports. Kim et. al. proposes three modifications to traditional rendering we collectively refer to as a *rotation-naive* (RN) warp:

- (1) Capture input(s) from the user at the latest possible moment
- (2) Apply these input(s) to the most recently rendered image, re-projecting the image to a novel view point
- (3) Allow the game to use the state from the resulting novel view point for simulation

When all three of the modifications above were applied in a relatively realistic situation, users showed an improvement in aiming time of around 80% of the penalty introduced by adding 80ms to the baseline system latency. Note that the RN implementation includes image artifacts, including distracting black borders from a lack of guard bands, yet a performance benefit exists even with this imperfect view. This work also presents data for theoretical or *oracle* warp conditions, instead rendering images directly from the correct view point, to show that beyond the most naive late warp, there is still room for a bit more aiming performance improvement. These implementations are referred to as rotation-oracle (RO) and translation-rotation-oracle (TRO) warps, and are compared to the RN warp in Figure 4.

In this work, we focus on items (1) and (2) from the list of RN modifications above, opting to provide an accessible demonstration of this technology that anyone can experience in a web browser at home. We feel that wide access to this demo is vital to motivating researchers and scientists with the understanding and ability to apply solutions like late warp to a wide variety of applications.

4 WEB DEMO

We developed a web-based application to demonstrate the effects of latency and late warp on aiming performance. This application

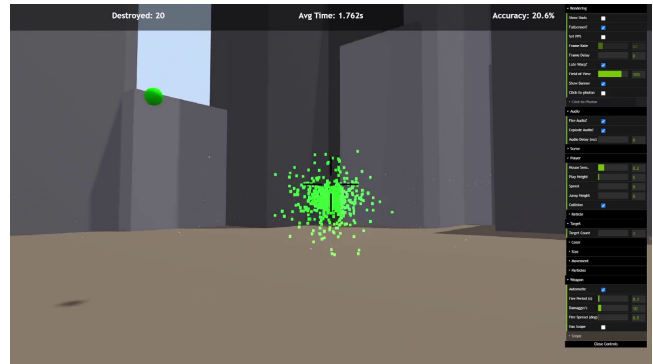


Figure 5: A screenshot of our demo app in sandbox mode. The green ball is a target, the black cross is the aiming reticle and the sandbox configuration UI is on the right.

uses Three.js (WebGL) for rendering, and provides an interactive FPS game environment where a series of targets must be eliminated as quickly as possible. It provides some basic aim training functionality similar to Aim Lab, KovaaK's FPS Aim Trainer and FirstPersonScience [Spjut et al. 2019b]. A screenshot of the application is provided in Figure 5.

In order to help users understand the impact of latency and benefits of late warp corrections, our demonstration includes three conditions where the only changes are the latency of the system and the application of a rudimentary, rotation-only late warp technique. The first condition is a baseline, which represents the lowest latency possible for the web application on the user's computer hardware. We provide a series of training trials in this condition to help familiarize users with the aiming system, adjust their mouse sensitivity, and allow them to acclimate to the native latency condition. The second condition adds approximately 80 ms of latency in an effort to emulate round trip time to a regional cloud rendering server. This demonstration shows that by adding cloud or other latency to the system, the ability of the player to efficiently hit targets is measurably reduced. In the third condition we "correct" the 80 ms of added end-to-end latency by running a rotation-only late warp on the delayed condition video with the most recent mouse input. This allows users to experience the impacts of latency as well as the potential performance gain offered by a simple warp technique. When run by one of the authors, the result was as seen in Figure 6.

4.1 Latency Injection in a Web Application

To mimic the effects of increased system latency in a web application we buffer and delay user inputs before altering game state using them. This increases the time from receiving an input to updating the scene for rendering based upon it. Typical Javascript input event handlers are used to route events into queues that are then processed on a per animation frame basis. We offer support for either allowing the browser to automatically call this animation function via `requestAnimationFrame()`, or attempting to manually time frames using the `setTimeout()` method. In cases of high system frame rates we suggest use of automatic, browser-based updates due to limitations in `setTimeout()` time granularity, and accordingly adopt this as our default setting for the experiment.

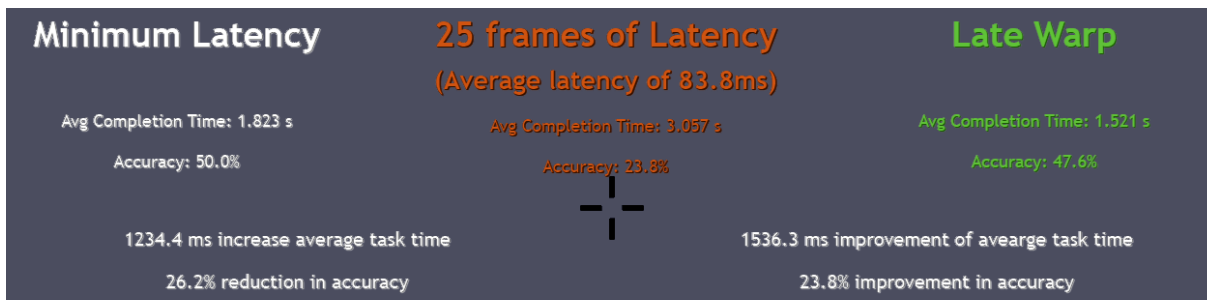


Figure 6: Demo results comparison for one of the authors.

4.2 Player Experience

When running the experiment users should make sure they have the latest updates to their web browser. We developed the application² to work with Chrome, and have tested that it works as intended in Chrome, Firefox and Edge on Windows 10.

Upon visiting the hosted page, or opening the HTML directly on their machine, the user enters the *experiment* mode of the application. In this mode they complete a brief training/adaptation session, followed by three different sessions, each containing an equal number of trials consisting of eliminating a single target. A session begins by presenting a red reference target, intended to reset the vertical aim direction, which when hit by the player begins the session's trials. Each trial consists of a single green target that moves along a linear path, the direction of which changes periodically (every 1-3s) or when it comes into contact with scene geometry. The player tries to eliminate targets as quickly as possible by translating the mouse to align the view direction indicator or *reticle* with the target, then clicking to eliminate it, spawning a new target. The banner at the top of the screen displays the remaining target count when in experiment mode. Once all targets are eliminated, a session is completed. When all sessions are completed the application displays a summary indicating the change in average completion time, accuracy, and actual latency between the sessions. By pressing shift at this summary the user enters *sandbox* mode, wherein they can customize many of the parameters included in this experiment including frame rate and delay (latency), use of a simple late warp, scene geometry, player height and motion, as well as target and weapon behavior. When in sandbox mode the banner at the top of the screen displays targets destroyed, average completion time, and overall accuracy and can be reset by pressing the R key at any time.

5 CONCLUSION

Reduced latency is beneficial for interactive applications, with the magnitude of benefit depending largely on the specific details of each application and its interaction. The FPS game genre was selected for this demonstration because of the known impact of latency on performance and its broadening appeal in the growing esports market. Our web-based demo enables users to experience the impacts of latency and benefits of late warp from the safety of their own home, even while the COVID-19 pandemic has prevented gathering the SIGGRAPH community this year. We hope

attendees will be inspired to consider latency in future work after experiencing this demo and reviewing the referenced user study results. We aspire to bring previously VR-specific techniques like late warp to a broader audience, enabling the next generation of graphics systems distributed across higher latency environments.

REFERENCES

- Elie Cattan, Amélie Rochet-Capellan, Pascal Perrier, and François Bérard. 2015. Reducing Latency with a Continuous Prediction: Effects on Users' Performance in Direct-Touch Target Acquisitions. In *Proceedings of the 2015 International Conference on Interactive Tabletops and Surfaces (Madeira, Portugal) (ITS '15)*. Association for Computing Machinery, New York, NY, USA, 205–214. <https://doi.org/10.1145/2817721.2817736>
- Paul M Fitts and James R Peterson. 1964. Information capacity of discrete motor responses. *Journal of experimental psychology* 67, 2 (1964), 103.
- Zenja Ivkovic, Ian Stavness, Carl Gutwin, and Steven Sutcliffe. 2015. *Quantifying and Mitigating the Negative Effects of Local Latencies on Aiming in 3D Shooter Games*. Association for Computing Machinery, New York, NY, USA, 135–144. <https://doi.org/10.1145/2702123.2702432>
- Ricardo Jota, Albert Ng, Paul Dietz, and Daniel Wigdor. 2013. *How Fast is Fast Enough? A Study of the Effects of Latency in Direct-Touch Pointing Tasks*. Association for Computing Machinery, New York, NY, USA, 2291–2300. <https://doi.org/10.1145/2470654.2481317>
- Joohwan Kim, Pyarelal Knowles, Josef Spjut, Ben Boudaoud, and Morgan McGuire. 2020. Post-Render Warp with Late Input Sampling Improves Aiming Under High Latency Conditions. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 3, 2 (2020), 1–18.
- Shengmei Liu, Atsuo Kuwahara, James Scovell, Jamie Sherman, and Mark Claypool. 2021. Comparing the Effects of Network Latency versus Local Latency on Competitive First Person Shooter Game Players. In *Esports and High-Performance HCI Workshop*. OSF Preprints, online, 4 pages. <https://doi.org/10.31219/osf.io/bhxxy>
- Julian Looser, Andy Cockburn, and Joshua Savage. 2005. On the validity of using First-Person Shooters for Fitts' law studies. *People and Computers XIX* 2 (2005), 33–36.
- I. Scott MacKenzie and Colin Ware. 1993. Lag as a Determinant of Human Performance in Interactive Systems. In *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems* (Amsterdam, The Netherlands) (CHI '93). Association for Computing Machinery, New York, NY, USA, 488–493. <https://doi.org/10.1145/169059.169431>
- Josef Spjut, Ben Boudaoud, Kamran Binaee, Jonghyun Kim, Alexander Majercik, Morgan McGuire, David Luebke, and Joohwan Kim. 2019a. Latency of 30 ms Benefits First Person Targeting Tasks More Than Refresh Rate Above 60 Hz. In *SIGGRAPH Asia 2019 Technical Briefs*. ACM, Brisbane, Australia, 110–113.
- Josef Spjut, Ben Boudaoud, Kamran Binaee, Alexander Majercik, Morgan McGuire, and Joohwan Kim. 2019b. FirstPersonScience: Quantifying Psychophysics for First Person Shooter Tasks. In *UCI Esports Conference*. UCI, Irvine, CA, 7 pages.
- Josef Spjut, Ben Boudaoud, and Joohwan Kim. 2021. A Case Study of First Person Aiming at Low Latency for Esports. In *Esports and High-Performance HCI Workshop*. OSF Preprints, online, 4 pages. <https://doi.org/10.31219/osf.io/nu9p3>
- R. J. Teather, A. Pavlovych, W. Stuerzlinger, and I. S. MacKenzie. 2009. Effects of tracking technology, latency, and spatial jitter on object movement. In *2009 IEEE Symposium on 3D User Interfaces*. IEEE, Lafayette, LA, USA, 43–50. <https://doi.org/10.1109/3DUI.2009.4811204>

²Available at <https://nvlabs.github.io/latewarp-s2021/>.